Informatique quantique 2

Quantum Approximate Optimization Algorithm

Valentin Gilbert – Quantum computing consultant at Qounselor

2025-2026

Plan

Part 1 :

1- Introduction to Variational Quantum Algorithms (VQA):

- General structure
- VQLS, VQE, QNN, QAOA

2- QAOA:

- Derivation from the Adiabatic Theorem
- Unitary implementation
- Optimization of the parameters
- Strengths and limitations

Part 2 :

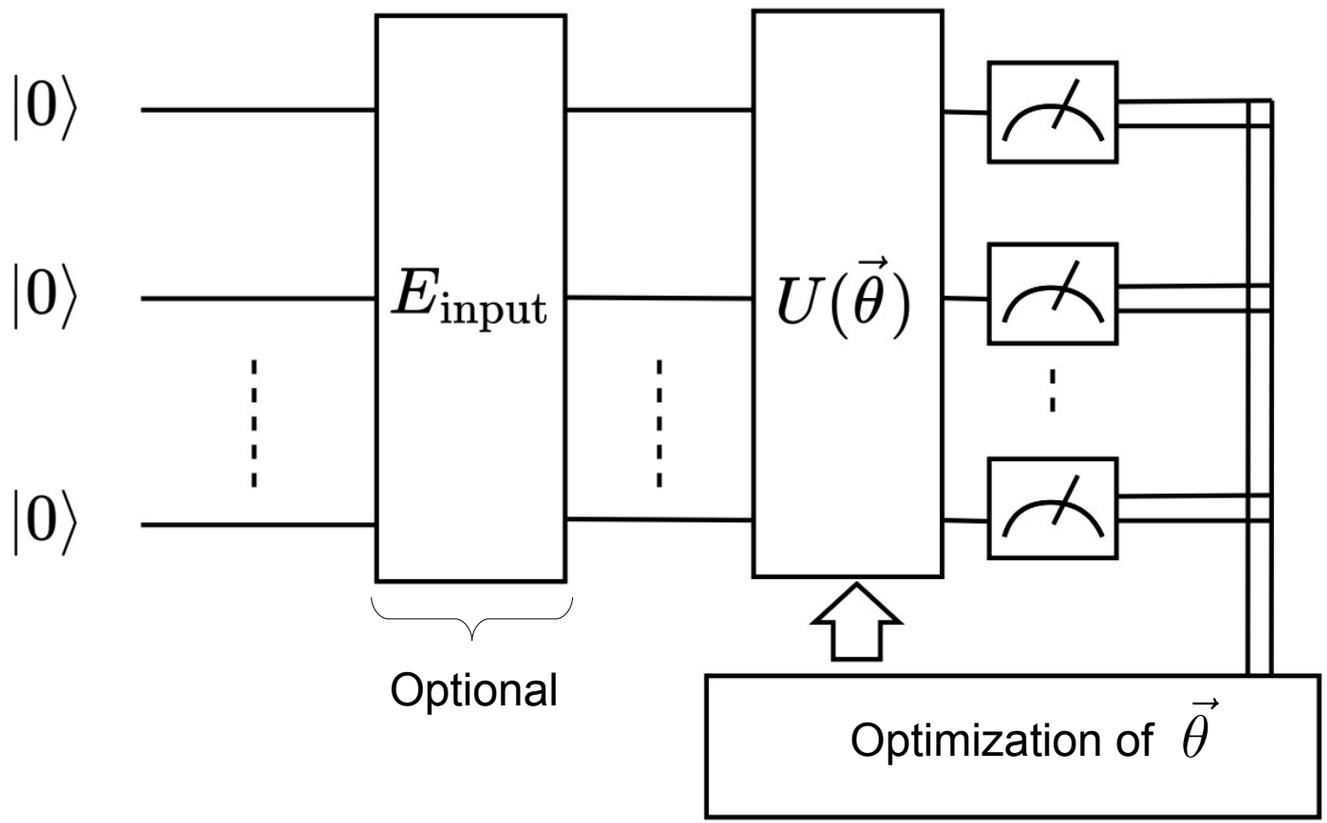
1- QAOA with other mixers:

- Mixer for the Maximum Independent Set problem
- Mixer for the TSP problem

Introduction to Variationnal Quantum Algorithm

Variational Quantum Algorithm

Variational gate-based (digital) quantum computer (universal):



Aim: Maximize a classical objective function that takes as input the measurements done on the quantum computer.

Variationnal Quantum Algorithm

Why do we use Variationnal Quantum Algorithms ?

- We are in NISQ (Noisy-Intermediate-Scale Quantum) era:
  - Existing universal quantum computers are **small** ($\simeq 100$ qubits)
  - Existing universal quantum computers are **noisy** ($\simeq 10^{-4}$ per multi-qubit operation in the best case)

=> We cannot run quantum algorithms with certified speed-ups (Grover, Shor etc.) because they require quantum error correction (i.e., Fault Tolerant Quantum Computing (FTQC)).

Requirements to provide interesting results with VQA and FTQC algorithms:

	VQA	FTQC
Number of physical qubits involved	> 100	> 10 000
Required fidelity of operations	$< 10^{-4}$	$< 10^{-6}$
Depth of quantum circuits	shallow	deep
Chip interconnection	more or less sparse	dense

Variationnal Quantum Algorithm (VQA)

<u>Applications solved with variational quantum algorithms:</u>

- Theoretical physics and chemistry problems (2014):
  - Variationnal Quantum Eigensolver (VQE) algorithm – A. Peruzzo et al.
    Link to the publication: https://pmc.ncbi.nlm.nih.gov/articles/PMC4124861/

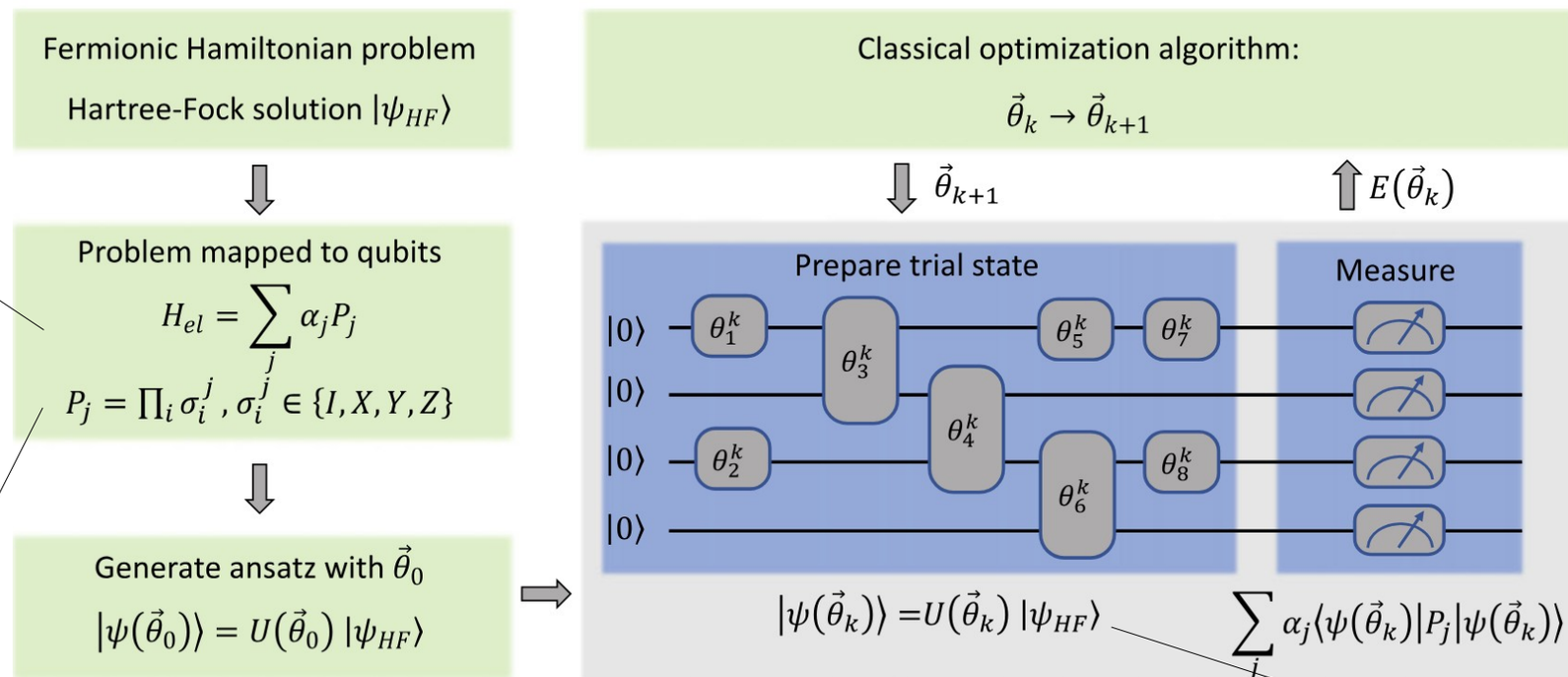
<u>Aim of the problem:</u> Find the ground state of a specific Hamiltonian (state of lowest energy)

The Hamiltonian of interest is decomposed into a suite of observables (the sum of different ways of observing your quantum system). The combination of the results of these observables gives you an estimation of the energy associated with a quantum state. If this energy is the minimum, you have the ground state.

The quantum system is then optimized to minimize the output energy (expectation value). The quantum system is used as a sort of proxy to find the ground state energy.

In the worst case, there are 4^n observables (meaning that it does not scale well)

The measurement of a single observable requires many executions of the quantum circuit.



Expectation value of a single Pauli operator.

The ansatz here is usually fixed and does not depend on H_{el} .

Source: https://link.springer.com/article/10.1186/s41313-021-00032-6

Variationnal Quantum Algorithm (VQA)

Applications solved with variational quantum algorithms:

- Linear System resolution (2019):
  - Variationnal Quantum Linear Solver (VQLS) algorithm – C. Bravo-Prieto et al. Link to the publication: https://arxiv.org/abs/1909.05820

Aim of the problem: Find a solution to a linear system of equations, which is equivalent to finding the vector $\vec{x}$ such that:

$$A\vec{x} = \vec{b} \qquad \text{encoded as} \qquad A\,|x(\theta)\rangle \propto |b\rangle$$

Circuit inputs:

- A, which must be easily encoded as a quantum circuit (not the case in general)
- The state $|b\rangle$ corresponding to the vector $\vec{b}$

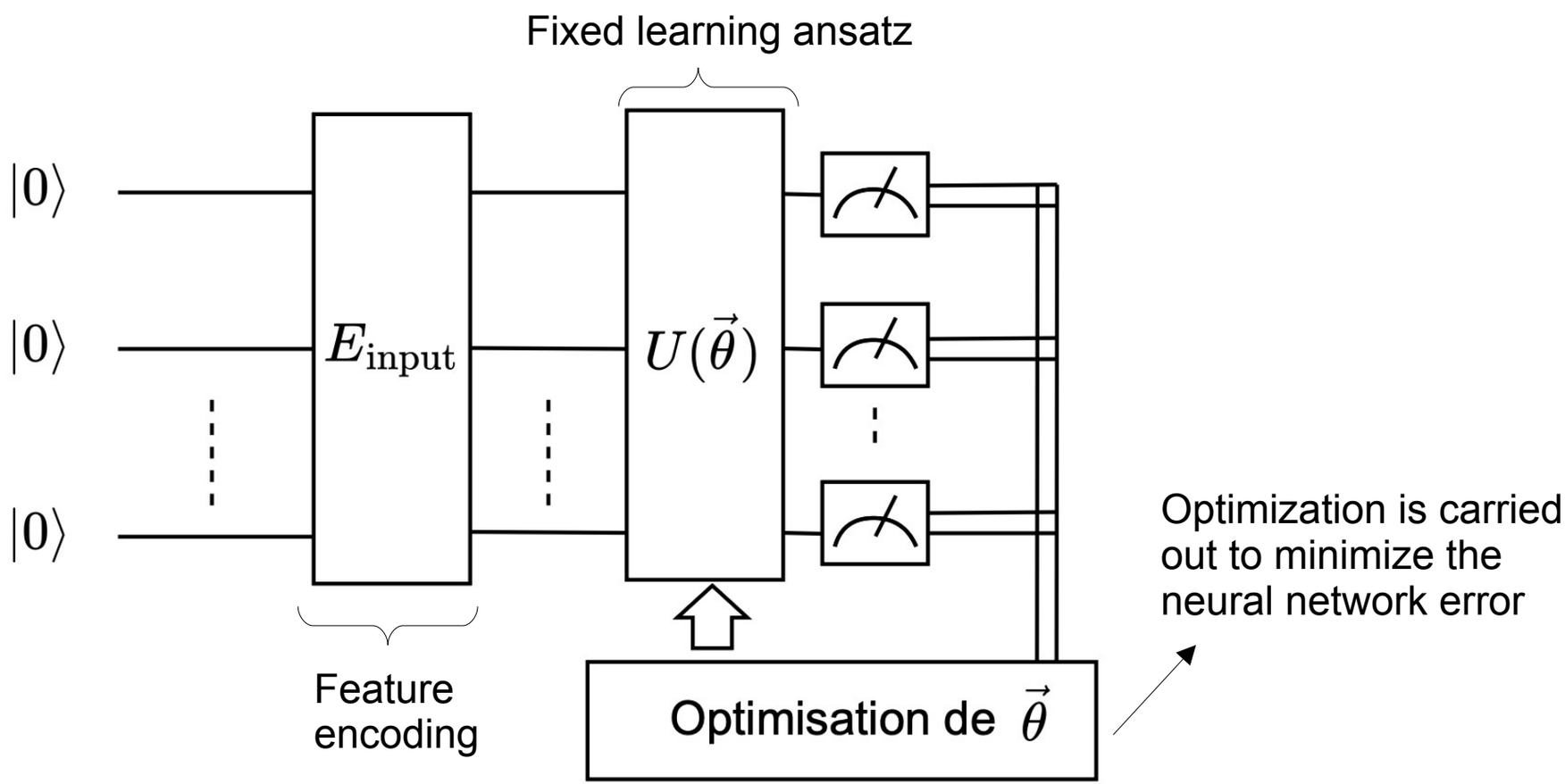
Aim of the variational loop:

- Minimize the a cost function C related to the inclination of $A\,|x(\theta)\rangle$ being proportional to $|b\rangle$.

Variationnal Quantum Algorithm (VQA)

<u>Applications solved with variational quantum algorithms:</u>

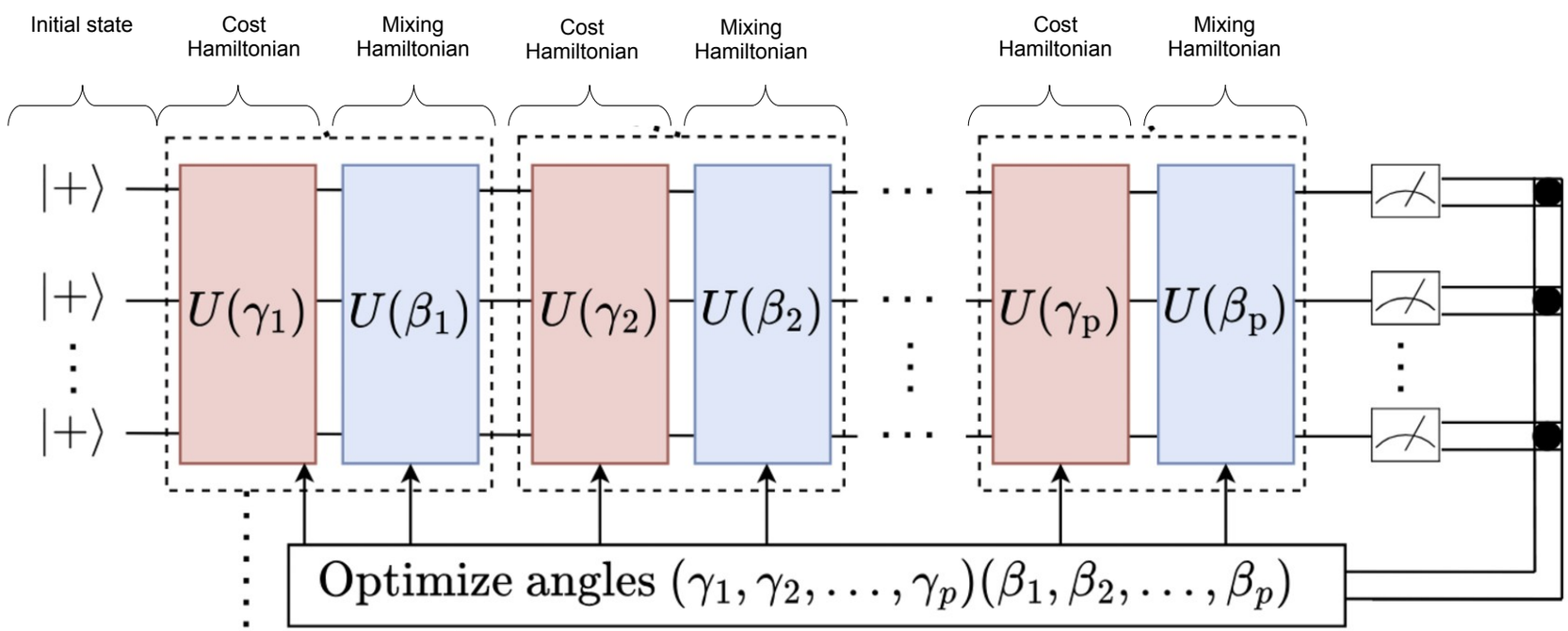
- Machine learning (1995):
  - Quantum Neural network – S. Kak
    Link to the publication: https://www.sciencedirect.com/science/article/abs/pii/002002559400095S



Variationnal Quantum Algorithm (VQA)

<u>Applications solved with variational quantum algorithms:</u>

- Optimization (2014):
  - The Quantum Approximate Optimization Algorithm (QAOA) – E. Farhi et al.
    Link to the publication: https://arxiv.org/abs/1411.4028



The optimization is done according to the output measured solutions that are re-injected into the cost function

The Quantum Approximate Optimization Algorithm

QAOA

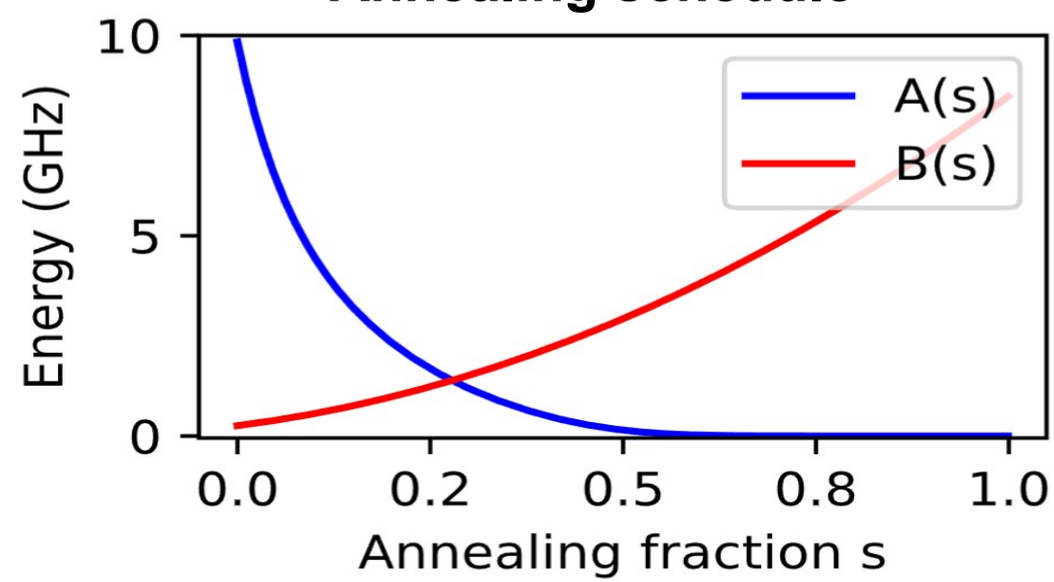
From Quantum Annealing (QA) to the QAOA:

D-Wave Transverse-Field Ising Model (TFIM):

$$s \in [0, 1]$$

$$H(s) = -\frac{A(s)}{2} H_{init} + \frac{B(s)}{2} H_{final}$$

Functions A(s) and B(s) are called the **Annealing schedule**



Transverse-field Hamiltonian:

$$H_{init} = \sum_{i=1}^{n} \sigma_i^x \qquad \sigma^x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Problem Hamiltonian :

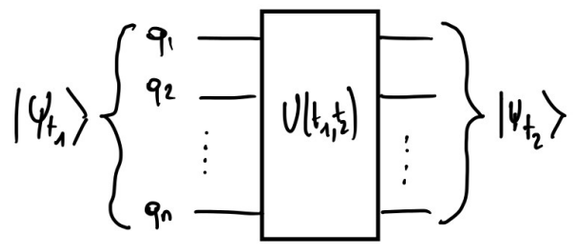
$$H_{final} = \sum_{i=1}^{n} h_i \sigma_i^z + \sum_{i<j} J_{i,j} \sigma_i^z \sigma_j^z$$

QAOA

From Quantum Annealing (QA) to the QAOA:

Evolution of Closed Quantum Systems (Postulates of quantum mechanic):

$$|\psi_{t_2}\rangle = U(t_1, t_2)\,|\psi_{t_1}\rangle$$



$|\psi_{t_i}\rangle$: n-qubit quantum state of size 2^n at time t_i
$U(t_1, t_2)$: Unitary matrix corresponding to a Hamiltonian evolution between t_1 and t_2

=> Gives the relation between two quantum states a closed system.

Refined version of the Evolution of Closed Quantum Systems (describe the system in contiuous time)
Shrödinger Equation:

$$i\frac{\partial\,|\psi(t)\rangle}{\partial t} = H\,|\psi(t)\rangle$$

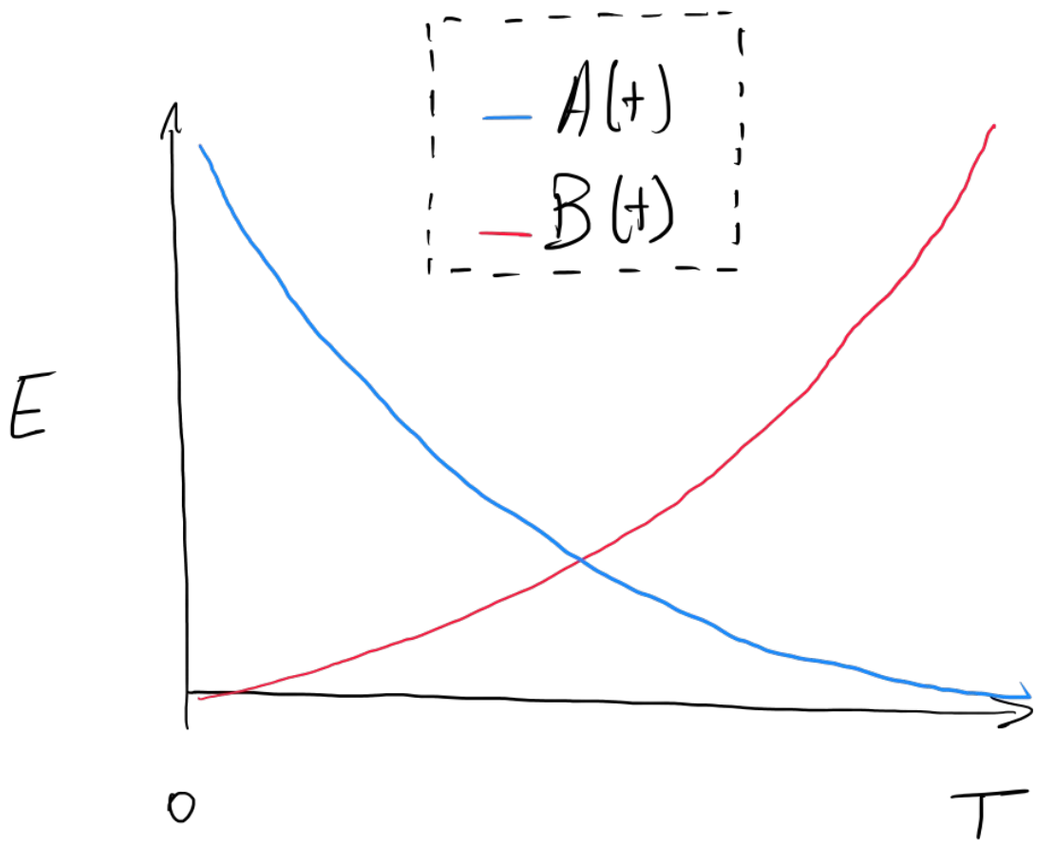
$|\psi(t)\rangle$: n-qubit quantum state of size 2^n
H : Hamiltonian of the system (Hermitian matrix)

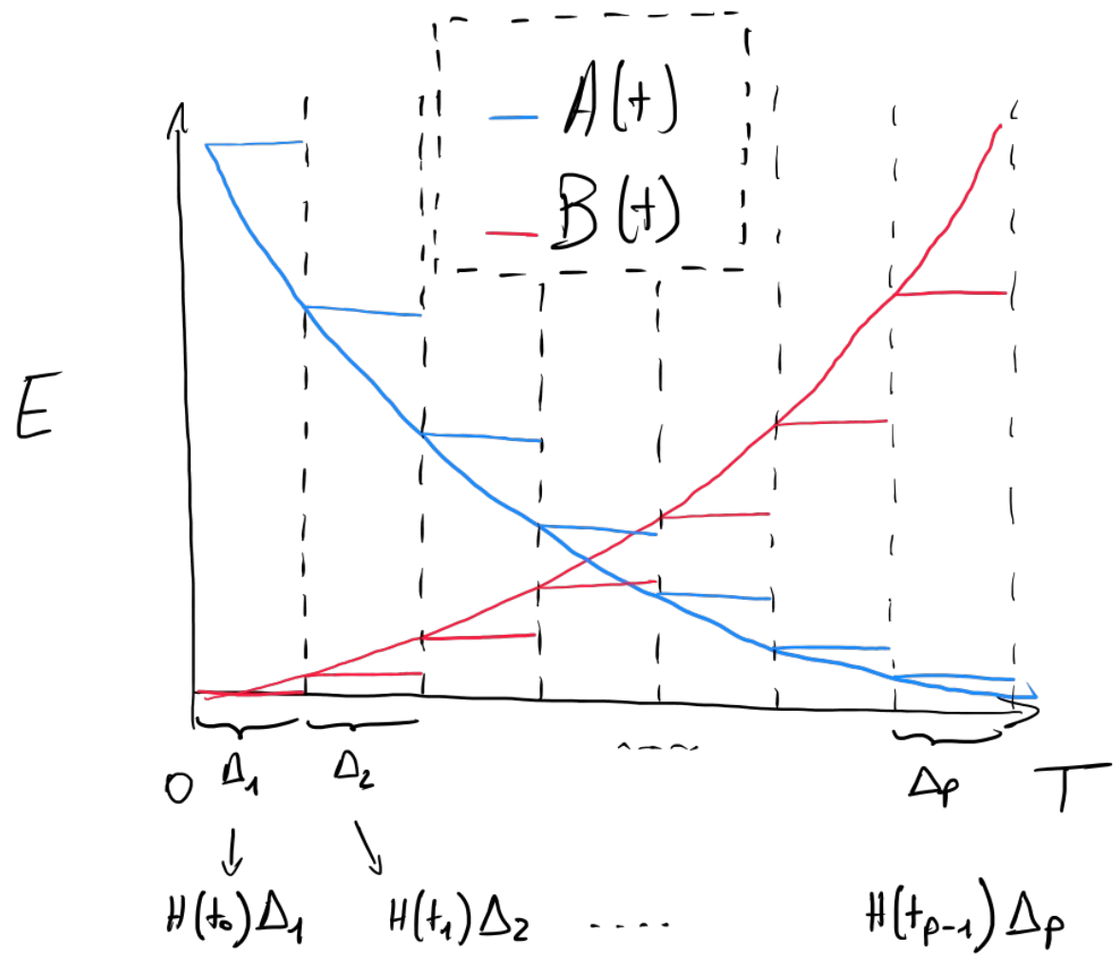
Solving Shrödinger Equation gives: $\boxed{|\psi_{t_2}\rangle = e^{-iH(t_2-t_1)}\,|\psi_{t_1}\rangle = U(t_1, t_2)\,|\psi_{t_1}\rangle}$

QAOA

<u>From Quantum Annealing (QA) to the QAOA:</u>

- **Step 1:** Approximate the continuous time evolution by p constant Hamiltonian evolution steps:





QAOA

<u>From Quantum Annealing (QA) to the QAOA:</u>

- **Step 1:** Approximate the continuous time evolution by p constant Hamiltonian evolution steps:

- Instead of having a continuous-time evolution we would then have:

$$|\psi(t_p)\rangle = e^{-iH(t_{\mathrm{p}-1})\Delta_p}\,|\psi(t_{\mathrm{p}-1})\rangle$$

- The whole evolution can then be characterized as:

$$|\psi(t_{\mathrm{p}})\rangle = \underbrace{e^{-iH(t_{p-1})\Delta_{\mathrm{p}}}}_{U(\Delta_{\mathrm{p}})}\underbrace{e^{-iH(t_{p-2})\Delta_{\mathrm{p}-1})}}_{U(\Delta_{\mathrm{p}-1})}....\underbrace{e^{-iH(t_1)\Delta_2}}_{U(\Delta_2)}\underbrace{e^{-iH(t_0)\Delta_1}}_{U(\Delta_1)}\,|\psi(t_0)\rangle$$

- Can we easily create $U(\Delta_k)$ from the exponential $e^{-iH(t_{k-1})\Delta_k}$?

QAOA

<u>Operator commutation and anti-commutation:</u>

Why is it important ?
Because if you have two unitaries describing a quantum evolution U_1 and U_2, the evolution described by the ordering $U_1 U_2$ and $U_2 U_1$ can lead to different results.

Given two matrices U_1 and U_2, these matrices are said to commute if:

$$[U_1, U_2] = U_1 U_2 - U_2 U_1 = 0$$

If U_1 and U_2commute, then $U_1 U_2$ and $U_2 U_1$ represent the same operation.

On the other hand, if U_1 and U_2 anti-commute, then:

$$[U_1, U_2] = U_1 U_2 - U_2 U_1 \neq 0$$

and their anti-commutator:

$$\{U_1, U_2\} = U_1 U_2 + U_2 U_1 = 0$$

QAOA

<u>From Quantum Annealing (QA) to the QAOA:</u>

- **Step 1:** Approximate the continuous time evolution by p constant Hamiltonian evolution steps:

- We recall that:

$$e^{-i(\Delta_k H(t_{k-1}))} = e^{-i\Delta_k (A(t_{k-1})H_{init} + B(t_{k-1})H_{final})}$$

- Can we easily implement the evolution e^{-itH} as a unitary matrix ?

If H can be decomposed such that $H = \sum_{i=1}^{l} H_i$ where all H_i operators commute, meaning that:

$$[H_i, H_j] = H_i H_j - H_j H_i = 0$$

Then, by the Baker–Campbell–Hausdorff formula, the following equality holds:

$$e^{-itH} = e^{-itH_1} e^{-itH_2} ... e^{-itH_l}$$

QAOA

From Quantum Annealing (QA) to the QAOA:

- **Step 1:** Approximate the continuous time evolution by p constant Hamiltonian evolution steps:

Exercise: Show that an evolution with a Hamiltonian composed of σ^x and σ^z operators cannot be efficiently expressed as a unitary transformation

$$\sigma^z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \qquad \sigma^x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

$$\begin{aligned}
[\sigma^x, \sigma^z] &= \sigma^x\sigma^z - \sigma^z\sigma^x \\
&= \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} - \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \\
&= \begin{pmatrix} 0 & -1 \\ 1 & 0- \end{pmatrix}\begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & -2 \\ 2 & 0 \end{pmatrix} \neq 0
\end{aligned}$$

=> X and Z operator do not commute.
We do not know efficient ways of expressing it as a unitary transformation.

QAOA

From Quantum Annealing (QA) to the QAOA:

- **Step 2:** Apply the first-order Suzuki-Trotter decomposition for each time step Δ_i .

Instead of trying the encode the exact quantum evolution as discrete steps, which is impossible due to the complexity of translating each Hamiltonian step in a unitary, we will use the following approximation:

- First-order Suzuki-Trotter decomposition:

$$e^{t(A+B)} = \lim_{n\to\infty} \left(e^{At/n} e^{Bt/n}\right)^n$$

- It means that if Δ_k is sufficiently small, we can write:

$$e^{-i(\Delta_k H(t_{k-1}))} = e^{-i\Delta_k (A(t_{k-1})H_{init} + B(t_{k-1})H_{final})}$$

$$\approx e^{-i\Delta_k A(t_{k-1})H_{init}} e^{-i\Delta_k B(t_{k-1})H_{final}}$$

QAOA

From Quantum Annealing (QA) to the QAOA:

- **Step 2:** Apply the first-order Suzuki-Trotter decomposition for each time step Δ_i .

Why step 2 in important ?

- Because we can now easily implement the unitary corresponding to each exponential evolution:

$$e^{-i\Delta_k A(t_{k-1})H_{init}}$$

$$e^{-i\Delta_k B(t_{k-1})H_{final}}$$

Exercise: Let's consider a problem with two qubits with:

$$H_{init} = \sigma_1^x + \sigma_2^x$$

$$H_{final} = \sigma_1^z \sigma_2^z + \sigma_1^z$$

Find the unitaries used to build the corresponding quantum circuit
Hint: Find usual quantum gates and their related exponential forms

QAOA

<u>Circuit implementation corresponding to the unitaries.</u>

- Implementation of the initial Hamiltonian:

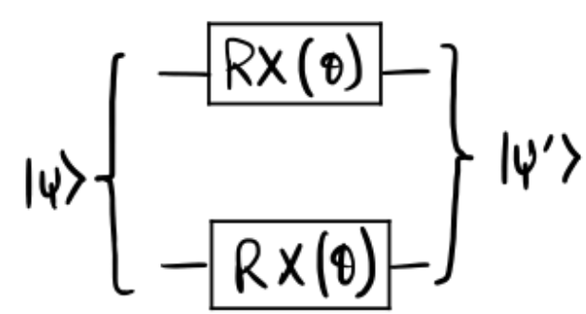
$$e^{-i\Delta_k A(t_{k-1}) H_{init}} = e^{-i\Delta_k A(t_{k-1})(\sigma_1^x + \sigma_2^x)}$$

(because σ_1^x and σ_2^x commute)

$$= e^{-i\underbrace{\Delta_k A(t_{k-1})}_{\beta_k}(\sigma_1^x)} e^{-i\underbrace{\Delta_k A(t_{k-1})}_{\beta_k}(\sigma_2^x)}$$

$$= \underbrace{e^{-i\beta_k \sigma_1^x} e^{-i\beta_k \sigma_2^x}}_{\text{Parametrized rotation gates}}$$

Corresponding quantum circuit:



$$RX(\theta) = e^{(-i\frac{\theta}{2}\sigma^x)} = \begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & -i\sin\left(\frac{\theta}{2}\right) \\ -i\sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}$$

QAOA

<u>Circuit implementation corresponding to the unitaries.</u>

- Implementation of the final Hamiltonian:

$$e^{-i\Delta_k B(t_{k-1})H_{init}} = e^{-i\Delta_k B(t_{k-1})(\sigma_1^z\sigma_2^z+\sigma_1^z)}$$

(The Hamiltonian terms are diagonal and then commute)

$$= e^{-\underbrace{i\Delta_k B(t_{k-1})}_{\gamma_k}(\sigma_1^z\sigma_2^z)} e^{-\underbrace{i\Delta_k B(t_{k-1})}_{\gamma_k}(\sigma_1^z)}$$

$$= e^{-i\gamma_k\sigma_1^z\sigma_2^z} \underbrace{e^{-i\gamma_k\sigma_1^z}}_{\text{Parametrized rotation gates}}$$

$$RZ(\phi) = e^{(-i\frac{\phi}{2}\sigma^z)} = \begin{pmatrix} e^{-i\frac{\phi}{2}} & 0 \\ 0 & e^{i\frac{\phi}{2}} \end{pmatrix}$$

QAOA

Circuit implementation corresponding to the unitaries.

- Implementation of the quadratic terms of the final Hamiltonian:

$$e^{-i\gamma_k \sigma_1^z \sigma_2^z}$$

Exercise: To find the unitary implementing the evolution first compute:

$$I_1 \otimes \sigma_2^z$$

$$\sigma_1^z \sigma_2^z$$

Does it help you ?

QAOA

<u>Circuit implementation corresponding to the unitaries.</u>

- Implementation of the final Hamiltonian:

$$e^{-i\gamma_k \sigma_1^z \sigma_2^z}$$

$$I_1 \otimes \sigma_2^z = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \otimes \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{array}{c} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \end{array} \begin{array}{l} |00\rangle \\ |01\rangle \\ |10\rangle \\ |11\rangle \end{array}$$

$$\sigma_1^z \sigma_2^z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \otimes \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{array}{l} |00\rangle \\ |01\rangle \\ |10\rangle \\ |11\rangle \end{array}$$

So these transformations are equivalent if we find some tricks and switch the amplitudes corresponding to |10> and |11> before doing the $I_1 \otimes \sigma_2^z$ operation

Can you think of a 2-qubit operation that could do that ?

QAOA

<u>Circuit implementation corresponding to the unitaries.</u>

- Implementation of the final Hamiltonian:

$$e^{-i\gamma_k \sigma_1^z \sigma_2^z}$$

- The CNOT gate can be used to implement the corresponding quantum circuit:

$$(CX_{1,2})(RZ_2)(CX_{1,2}) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

$$= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \sigma_1^z \sigma_2^z$$

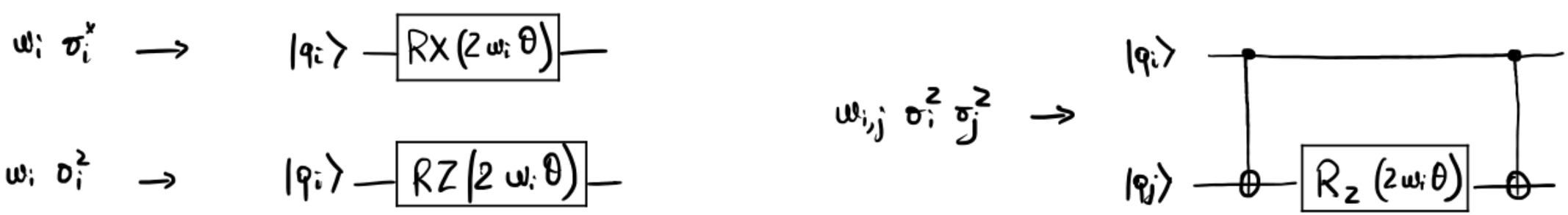
QAOA

<u>Circuit implementation corresponding to the unitaries.</u>

- Implementation recap given the linear-interpolated Hamiltonian:

$$H(s) = A(s)H_{init} + B(s)H_{final}$$

With only sum of σ_i^x operators in H_{init} and sum of single σ_i^z quadratic $\sigma_i^z \sigma_j^z$ in H_{final}, we can build a layer of QAOA corresponding to a Δ_i time step by substituting each term by:



QAOA

Advantage of the QAOA over AQC:

- Ability to shorten the adiabatic quantum evolution (by doing less layers)
- Generate shallow circuit (hence robust to quantum noise):
  - O(1) depth for implementing a layer corresponding to H_{init}
  - O(n) depth for implementing a layer corresponding to H_{final}
- Some scaling advantage compared to classical algorithms on very specific instances of problems: “Lower bounding the MaxCut of high girth 3-regular graphs using the QAOA” 2026 https://arxiv.org/abs/2503.12789 (E. Farhi et al.)
- Naturally scale well to higher-order terms $\sigma_i^z \sigma_j^z \sigma_k^z$ compared to QA devices.

Disadvantage of the QAOA over AQC:

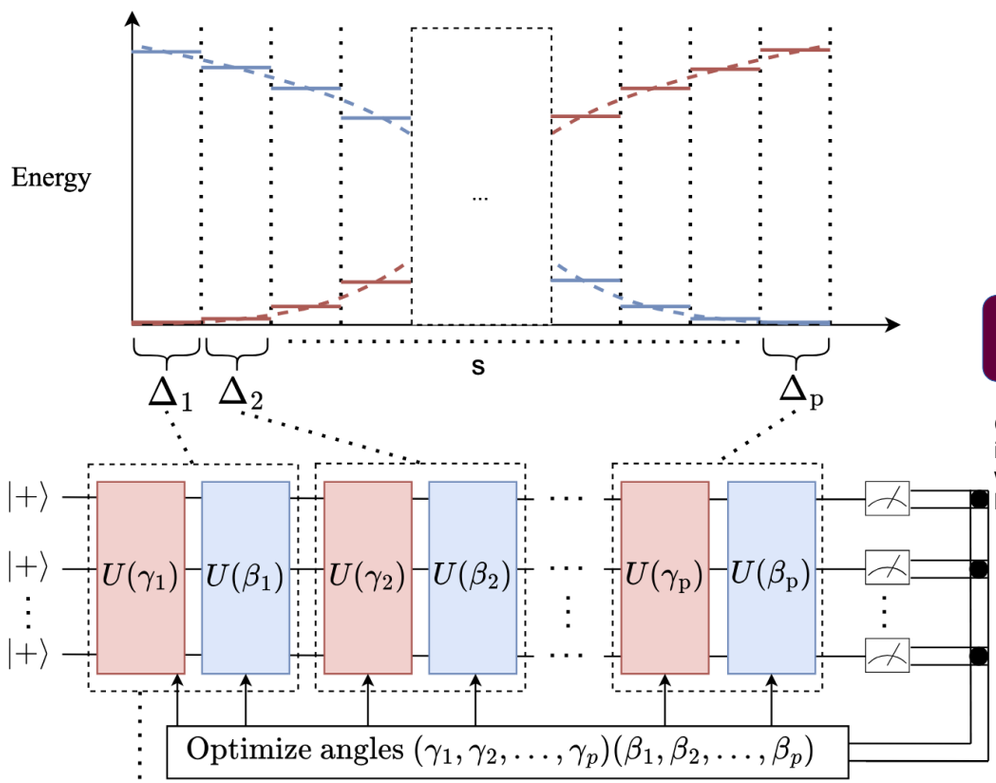
- Require optimization of angles β_i and γ_i , albeit these could be derived from the adiabatic function schedule. In practice, the number of QAOA layer
- Circuits gets deeper as H_{init} and H_{final} become more complex.
- Require a universal machine. The problem encoded with a QAOA is then usually smaller than the one that could be directly mapped to a Quantum Annealer.

QAOA

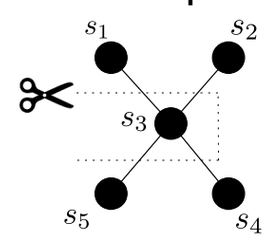
QAOA global implementation.

As QAOA seems to be derived from QA, what should be the initial quantum state ?

Multi-layer QAOA:



Example with a simple Ising instance (corresponding to the max-cut problem with the following graph)

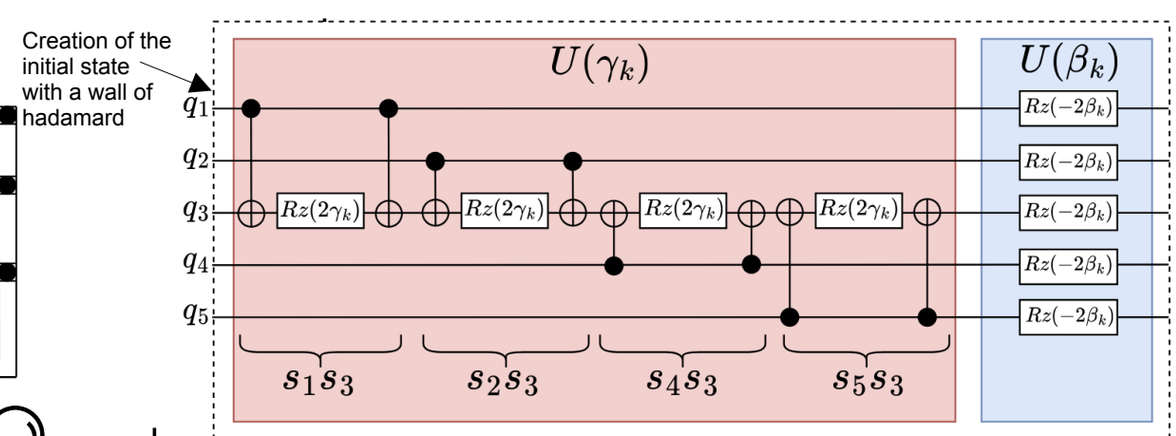


$$C_{Ising}(\mathbf{s}) = s_1 s_3 + s_2 s_3 + s_5 s_3 + s_4 s_3$$

$$H_{init} = \sigma_1^x + \sigma_2^x + \sigma_3^x + \sigma_4^x + \sigma_5^x$$

$$H_{final} = \sigma_1^z \sigma_3^z + \sigma_2^z \sigma_3^z + \sigma_5^z \sigma_3^z + \sigma_4^z \sigma_3^z$$

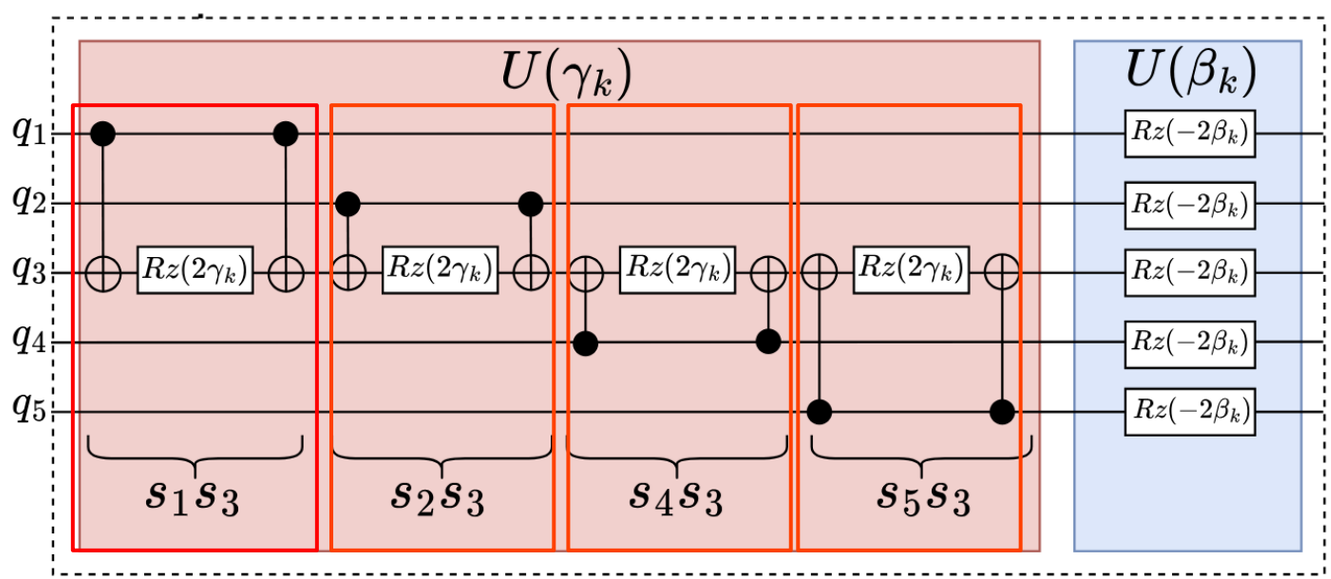
Create a depth-one QAOA circuit for this instance



QAOA

<u>Ordering of gates in each parametrized unitary:</u>

$$U(\gamma_k) = \underbrace{e^{-i\gamma_k(\sigma_1^z\sigma_3^z)}e^{-i\gamma_k(\sigma_2^z\sigma_3^z)}e^{-i\gamma_k(\sigma_4^z\sigma_3^z)}e^{-i\gamma_k(\sigma_5^z\sigma_3^z)}}_{\text{All the terms commute so the order is not important}}$$



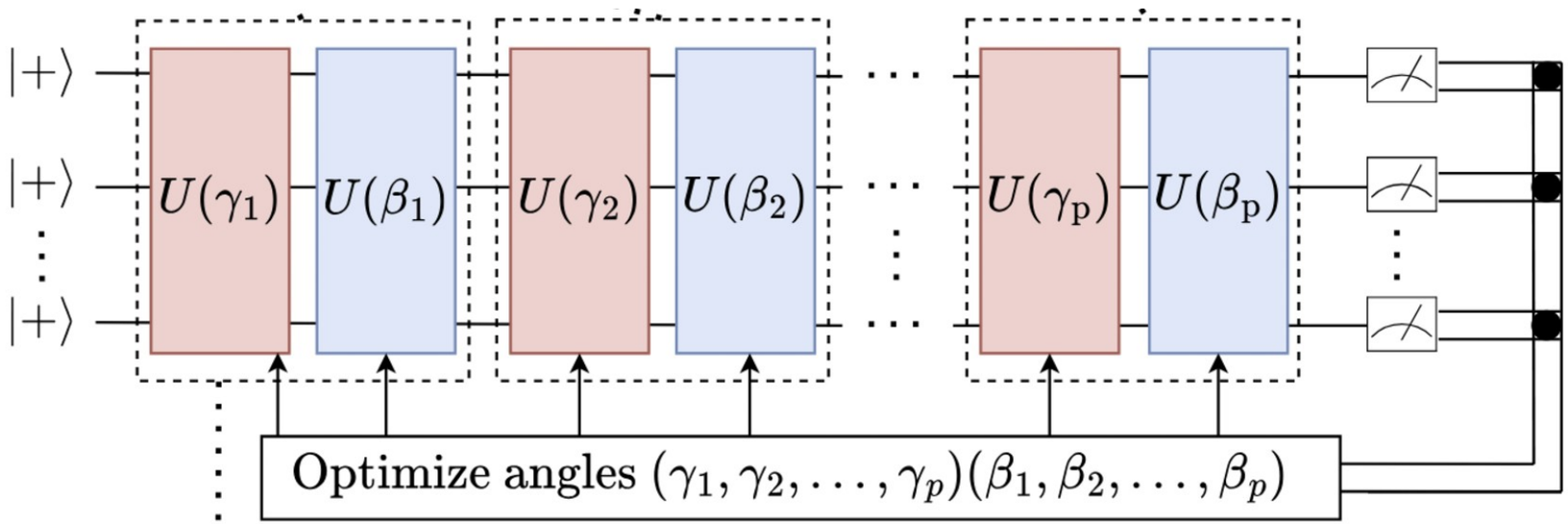
=> Each block can be freely ordered as wished (may impact the compilation of the quantum circuit and Hence, the overall depth of the quantum circuit)

QAOA

<u>Unitary parameters:</u>

- Quantum circuit structure
  Example with some Ising function

$$C_{Ising}(\mathbf{s}) = \sum_{i=1}^{n} h_i s_i + \sum_{i<j}^{n} J_{i,j} s_i s_j \qquad H_{final} = \sum_{i=1}^{n} h_i \sigma_i^z + \sum_{i<j} J_{i,j} \sigma_i^z \sigma_j^z$$



- The angles are optimized according to the expectation value: $\langle \psi_{QAOA} | H_{final} | \psi_{QAOA} \rangle$

=> We just measure the output of l runs, then evaluate the Ising cost function for the l outputs and compute the average.

QAOA

Unitary parameters:

- One angle per layer (when the number of steps is infinite, the angle can be directly derived from the value of the annealing schedule).
- Angles are gate rotations around the Bloch sphere. These gates should be able to access any configuration of the Bloch sphere.

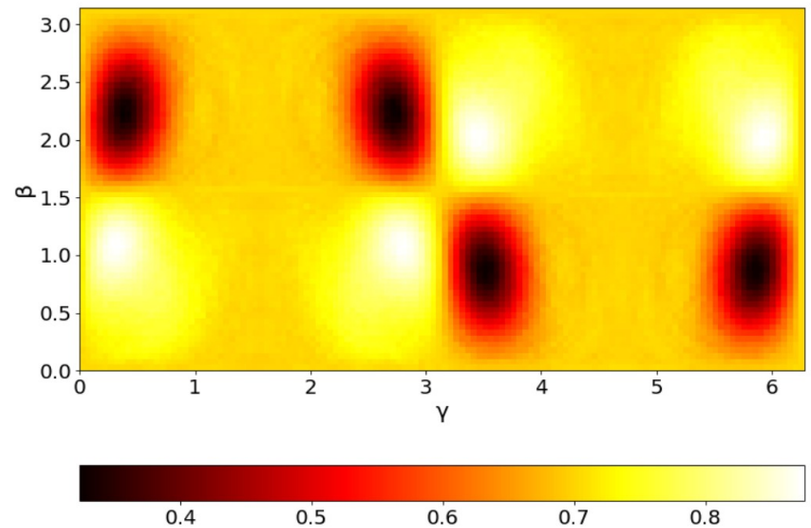
$$\left.\begin{aligned} \gamma_k &\in [0, 2\pi[\\ \beta_k &\in [0, 2\pi[\end{aligned}\right\} \text{Only valid if Ising weights } w_i, w_{i,j} > 1$$

- In practice, this range can be reduced depending on the mixers and potential problem symmetries.

Can you think of a problem we saw that has always symmetries in the cost of its solutions ?

- Optimization of the parameters:
  - Complex task to carry: the convergence to local optima requires many evaluations of the quantum circuit.
  - NP-hardness: https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.127.120502
  - Hardness amplified by noise.

QAOA



Value of the cost function

<u>Different optimization methods:</u>

- Grid search optimization:

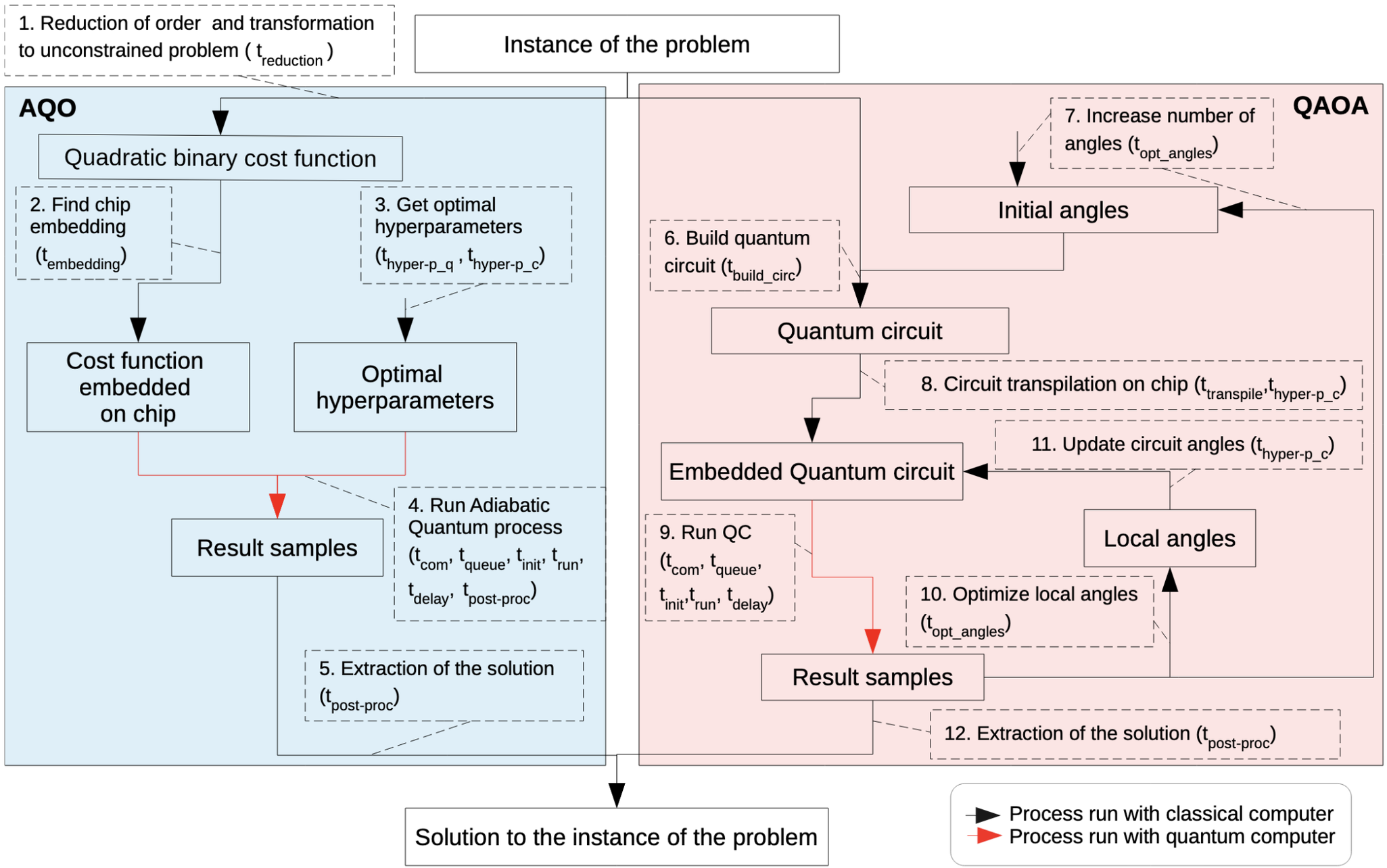
$$\gamma_k \in [0, 2\pi[$$

$$\beta_k \in [0, 2\pi[$$

- One shot optimization:
  - Generate a QAOA circuit with depth p.
  - Create a list of real $(\gamma_1, \gamma_2, \ldots. \gamma_p, \beta_1, \beta_2, \ldots \beta_p)$
  - Use a derivative-free method to optimize all the angles at the same time.
    You can check scipy python functions available at:
    scipy.optimize.minimize: https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize.html

- Iterative optimization strategies:
  - Linear interpolation of angles
  - Fourier optimization
    Source: https://journals.aps.org/prx/abstract/10.1103/PhysRevX.10.021067

QAOA vs QA workflow



Stengths and limitations of Variational Quantum Algorithms

Limitations

Barren Plateau

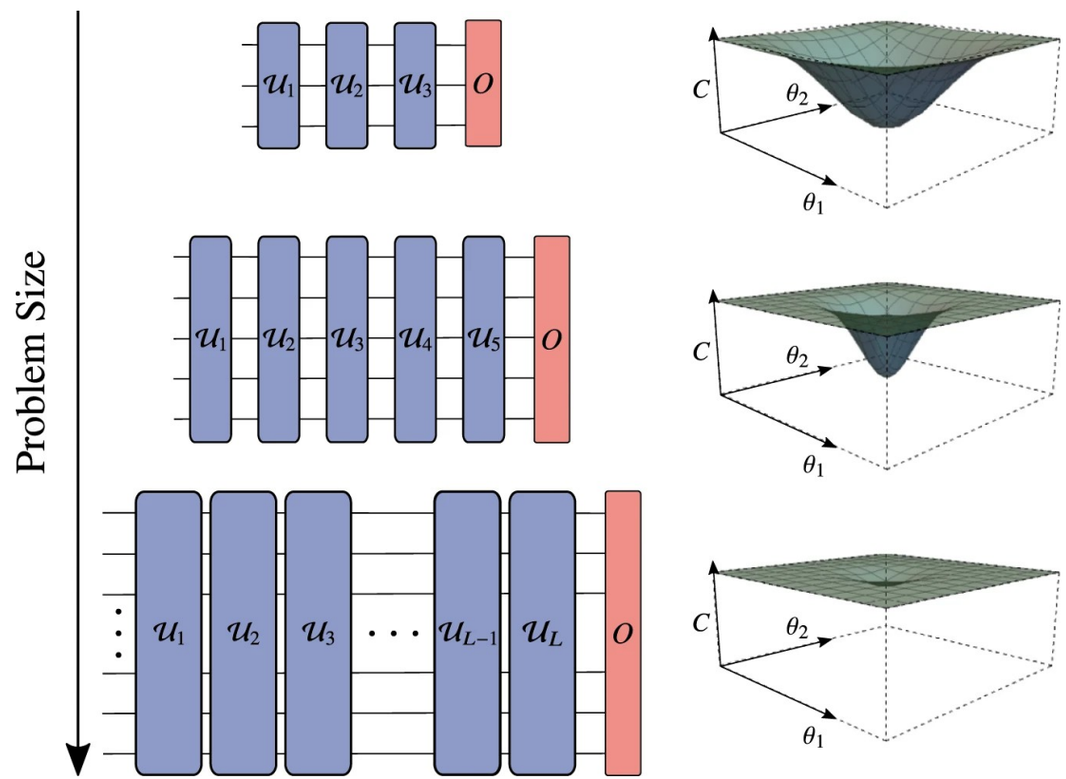
<u>As for standard neural network training variational quantum circuits is difficult:</u>

Publication discussing this phenomena:
https://www.nature.com/articles/s42254-025-00813-9

<u>Sources of Barren plateau:</u>

- Number of parameters in the variational quantum algorithm
- Noise on the quantum machine

=> We currently do not know if the optimization of large circuit will scale well.



Source: https://en.wikipedia.org/wiki/Quantum_neural_network#/media/File:Barren_plateaus_of_VQA.webp

Limitations

Parameter Optimization time

The time of optimization might be impacted by:

- Noise (hence depth of the quantum circuit)
- Gate time, gate set and initialization frequency of the quantum chip (e.g., Trapped-Ions vs Superconducting qubits)
- Speed of interconnection between the quantum computer and the classical computer

Disability to efficiently encode operations required by the quantum circuit

- VQLS: no efficient way to easily encode large matrices
- Neural network: no efficient way to encode input data in the neural network (unless the input data is already quantum => restrict the panel of potential use cases)

Number of required measurements

- VQE: requires several measurements to evaluate only a single observable (Pauli operator). Running VQE on large instances would require at least 10^8 shots to estimate only single points of energy (and then require optimization steps).
- In general the number of required measurements increases with noise to reduce variations in the observed results.

Strengths

Depth scaling of the quantum circuit:
For some simple problems the depth of the quantum circuit scales well:

- For Ising problems a QAOA layer is implemented with only a single RX for each qubit, 2 CNOT and one RZ for each quadratic term and one RZ for each single term.
- The QAOA can easily scale to higher-order terms $\sigma_i^z \sigma_j^z \sigma_k^z$ (it does not require reduction order methods as for analog quantum computer)

Some hope about the scaling of the method:

- On very specific instances (e.g. 3-regular graphs), there has been some proofs showing that the QAOA could outperform the approximation ratio provided by classical methods at constant depth (https://arxiv.org/abs/2503.12789)
- Scaling study about the QAOA (evaluation of the performance of the QAOA when instances grow in size) also show encouraging results by demonstrating that potential quantum advantage could appear using these methods. (https://www.science.org/doi/10.1126/sciadv.adm6761)

These studies generally underestimate the presence of noise on the quantum chips which could lead to disappointing results

Other QAOA mixers

Other QAOA mixers (MIS)

Motivation for inventing new mixers:

- Restrict the search space to solutions that are valid (e.g., respecting constraints).
- Increase expressivity of the QAOA (knowledge of the optimization problem).
- Facilitate the optimization of QAOA angles.

Up to now we saw the mixer:

$$H_{init} = \sum_{i=1}^{n} \sigma_i^x$$

Limitations of the Transverse-field mixing Hamiltonian:

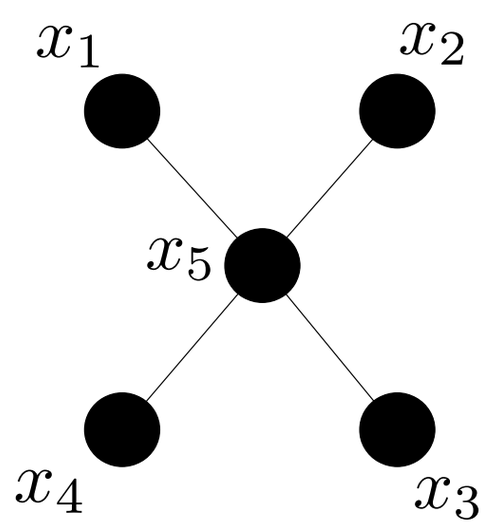
- All the solutions are considered as valid
- Encoding complex problems with constraints (e.g., Knapsack problem, MIS…) require the use of penalties if we want to
  => This makes unfeasible solutions feasible in practice.

Other QAOA mixers (MIS)

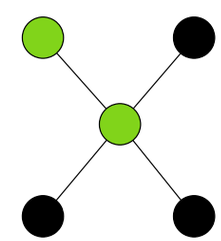
Example with the Maximum Independent Set:

$G = (V, E)$

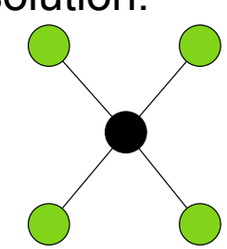
Problem: Find a subset of nodes of maximum size such that two neighboring nodes are not in this set.



Solution that is not valid:



Optimal solution:



Formulation of the problem:

$$x_v \in \{0, 1\} \qquad \max C_{MIS} = \sum_{v \in V} x_v \qquad \text{such that } x_u \neq x_v \text{ if } (u, v) \in E$$

Other QAOA mixers (MIS)

<u>Example with the Maximum Independent Set:</u>

<u>Solution 1:</u> Transform the constraint as a penalty inserted in the cost function and do not change the mixer:

$$\max C_{MIS} = \underbrace{\sum_{v \in V} x_v}_{\text{Maximizes the number of nodes in the independent set}} - \underbrace{2 \sum_{(u,v) \in E} x_u x_v}_{\text{Replaces the constraint by a fixed penalty}}$$

=> We could then use this cost function to generate the Ising cost function and then solve it with a standard QAOA.

=> We preserve the optimal solution.
Issue: we loose some information about the structure of the problem (some invalid solutions become valid due to the penalty which do not encode a **hard constraint**.

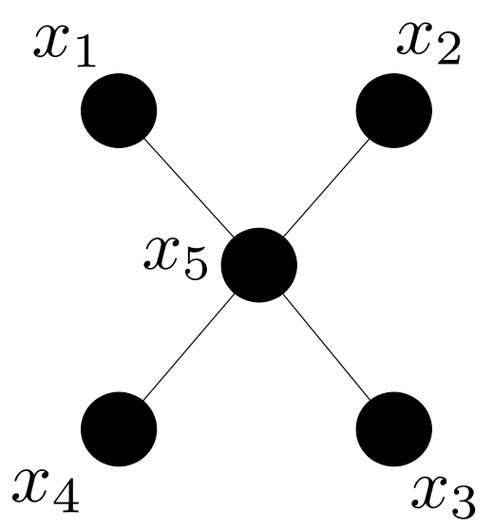
Other QAOA mixers (MIS)

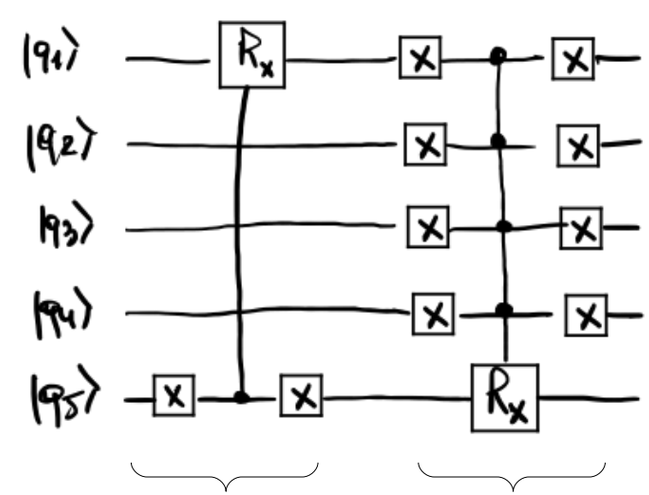
Example with the Maximum Independent Set:

Solution 2: Enforce the constraint in the mixer and change the Hamiltonian H_m (H_{init}). The idea is to apply a controlled rotation over the qubit q_i representing a node x_i only if the neighbors of the node x_i are not in the independent set.

Can you think of an operator that would permit you to do that ?

Implementation example for the following graph:





Other QAOA mixers (MIS)

<u>Example with the Maximum Independent Set:</u>

<u>Solution 2:</u> The problem Hamiltonian H_P (H_{final}) then simply corresponds to the cost function:

$$\max C_{MIS} = \sum_{v \in V} x_v$$

Find the unitary implementing H_P ?

We just have to substitute the variables as:

$$x_i \rightarrow \frac{1 - s_i}{2}$$

The unitary will then implement only evolutions looking like $\sum_{i=1}^{n} e^{-it\sigma_i^z}$

which can be implemented with a single wall of rotation gates around the Z axis.

Other QAOA mixers (TSP)

Example with the Travelling Salesman Problem:

"*Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?*"

The set of solutions is the set of all the permutations:

$$S_n = \{[\mathrm{city}_1, \mathrm{city}_2, \mathrm{city}_3, ...\mathrm{city}_n], [\mathrm{city}_2, \mathrm{city}_1, \mathrm{city}_3, ...\mathrm{city}_n], [\mathrm{city}_2, \mathrm{city}_3, \mathrm{city}_1, ...\mathrm{city}_n], ...\}$$

Find a binary formulation of this problem
- Domain
- Constraints
- Cost function

Other QAOA mixers (TSP)

<u>Example with the Travelling Salesman Problem:</u>

Formulation of the problem:

Domain: $x_{i,p} \in \{0, 1\}$

$x_{i,p}$ is set to one if the city i is visited at the step p of the tour. For n cities, there will be n^2 variables $x_{i,p}$.

Set of distances between city i and j : $d_{i,j} \in \mathbb{R}$

<u>Constraints:</u>

- There is only one city visited per position:

  for any fixed p the following equality must hold: $$\sum_{i=1}^{n} x_{i,p} = 1$$

- There is only one position per city:

  for any fixed i the following equality must hold: $$\sum_{p=1}^{n} x_{i,p} = 1$$

Other QAOA mixers (TSP)

<u>Example with the Travelling Salesman Problem:</u>

We then try to minimize the following cost function:

$$\min \sum_{p=1}^{n} \sum_{i,j} d_{i,j} x_{i,p} x_{j,p+1}$$

This cost function can then be used to implement the Hamiltonian H_P with the variable substitution:

$$x_{i,p} \rightarrow \frac{1 - s_{i,p}}{2}$$

Other QAOA mixers (TSP)

<u>Example with the Travelling Salesman Problem:</u>

Enforcing the permutation constraint with the Mixing Hamiltonian H_M :

$$H_M = \sum_{a,b} (X_a X_b + Y_a Y_b)$$

Show that this Hamiltonian only allow swapping interactions and preserve the number of ones.

Remember that:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \qquad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$$

Other QAOA mixers (TSP)

<u>Example with the Travelling Salesman Problem:</u>

Development of the computations:

Let's take two qubits and implement the desired Hamiltonian:

$$X_1X_2 + Y_1Y_2$$

$$X_1X_2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \otimes \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix} \quad Y_1Y_2 = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \otimes \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \end{pmatrix}$$

$$X_1X_2 + Y_1Y_2 = \begin{array}{c} \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \end{array} \begin{array}{l} |00\rangle \\ |01\rangle \\ |10\rangle \\ |11\rangle \end{array}$$

This operation allows only swapping states |01> and |10>

=> If we start with a random initial configuration, we will stay in a feasible space owing to the XX+YY mixer.

Universality of Adiabatic Quantum Computing

Universality of AQC

Quantum annealers are not universal:

$$H(s) = -\frac{A(s)}{2} H_{init} + \frac{B(s)}{2} H_{final}$$

$$H_{init} = \sum_{i=1}^{n} \sigma_i^x$$

$$H_{final} = \sum_{i=1}^{n} h_i \sigma_i^z + \sum_{i<j} J_{i,j} \sigma_i^z \sigma_j^z$$

Why Quantum Annealers are not universal ?

- Because the Hamiltonian H_{init} and H_{final} cannot be programmed as much as we would like (for example, we could not use the previously seen more advanced mixers, especially the one for the TSP)
- The QAOA algorithm is more versatile as it enables programming richer interactions between qubits (we cannot compare #qubits in analog and gate-based models)

Thank you for your attention !